Database connections can be *physical* or *server* connections; server connections are used by the client Ruby on Rails application with the Active Record Connection Pool when it opens connections.

When a proxy server is between Ruby on Rails and PostgreSQL, connections from the client application to the proxy server are called *client* connections. Client connections between Rails and the proxy are re-used. Multiple client connections may use a single physical connection. Server connections between the proxy and PostgreSQL can be used more efficiently by client applications. You haven't yet worked with a proxy server but you'll see more about that soon.

Active Record lazily allocates physical database connections, meaning they're opened as needed and then kept open in an `idle` state when a query completes.

As a brief refresher on the backend pids and PostgreSQL processes, for new database connections the master PostgreSQL process is forked. Process forking is fast, but does add latency to the client-server round trip time. To eliminate this latency, the application layer connection pooler will keep connections open but idling. This way the process forking latency is not incurred for every new query. This is kind of like leaving your car running when you go into a store for just a minute!

In the next section, you'll look at dedicated connection pooling software that enhances the capabilities beyond what's possible with the Active Record Connection. These capabilities help your application reach higher levels of concurrency beyond what would otherwise be possible.

Running Out Of Connections

Ruby on Rails application instances can be scaled horizontally by running more instances of the (stateless) application. Commonly, background job processing with Sidekiq[8] is run with multiple processes as well, and these background workers also require database connections.

For puma or Sidekiq, more processes are added with multiple threads of execution, and each thread requires a database connection.

Client queries may overrun the maximum number of database connections. This is called *exhausting* the connections. When this happens, no new queries can run because no connections are available. The server will return an error to the client like `FATAL: sorry, too many clients already`.

8. https://github.com/mperham/sidekiq